

WESAAC 14 Tutorial

Programming Agent Systems Exercise

[João Leite](#)

Universidade Nova de Lisboa

(course slides available [here](#))

This programming exercise will illustrate some of the features of Jason through the development and refinement of the Gold Miners implementation.

In each of the following items, we start with a general description and its objective, a description of the setting to test the exercise, followed by some brief explanation of the steps involved in achieving it and some hints on how to do it. Depending on your programming experience, you may decide to only read the introduction and objective to develop your own original solution, or also read the explanation and hints.

Group I

To get started, you can download the Jason project from [here](#). Then, after unzipping it into some directory, open Eclipse and import the downloaded project (File -> Import -> Jason Project -> Pick directory -> Select Project GoldMiners). Take a few minutes to understand the miner.asl file. You can test it using environment id=1; leader; miner; dummy; dummy; dummy (the .mas2j file already encodes this configuration).

a)

Introduction: To help understand what is going on, and what the agents are doing, it is useful to have agents print messages on the console. This is particularly useful when agents are moving to random positions that we cannot anticipate.

Objective: modify the current miner.asl specification file in a way that the miners print a message on the console announcing the location they randomly picked to go near.

Test: environment id=1; leader;miner;dummy;dummy;dummy

Description: the objective can be accomplished by changing the plan where the next location is chosen, to include an additional action that prints the desired message.

Hints: recall that there is an internal action `.print(term1,term2,...)`.

Solution: available [here](#).

b)

Introduction: One of the reasons why the miner is going **near** a cell, and not **to** a cell, is because it may happen that the randomly chosen cell is not reachable e.g. because there are walls blocking any path to it. The current implementation already deals with this by adding the belief `near(X,Y)` whenever the agent cannot reach the cell, or whether it is already in the vicinity. However, we are not aware of which of these two reasons actually happened.

Objective: modify the current miner.asl specification file in a way that the miners print a message on the console announcing either that they have reached the neighborhood of the target cell, or that the target cell is not reachable, depending on what is the actual cause.

Test: environment id=2; leader;miner;dummy;dummy;dummy

Description: the objective can be accomplished by changing the plans for the goal **near(X,Y)** so that the specific message is executed.

Hints: you might have to split one of the plans in two, so that you can differentiate the context, on which the message to be printed depends.

Solution: available [here](#).

Group II

With the previous implementation, the miner was not really mining. It was only wondering around. You can see for yourself by trying it with environment id=3.

If you now use the file [miner1.asl](#) instead, you will see that several plans have been added to deal with the mining of gold. Take a few minutes to understand the new miner1.asl file. Most parts are commented to help you understand. Try it with environment id=3; leader;miner1;dummy;dummy;dummy.

Note: if you simply copied the code of miner1.asl into your previous miner.asl agent, then the configuration should refer to miner and not miner1. From now on, we will assume that you have copied the code in miner1.asl into your miner.asl file.

c)

Introduction: Whereas so far we have only considered one miner, the miner will have to share its environment with other agents specialized in picking up gold and dropping it on the depot. To know who drops more gold pieces on the depot, we will trust agents to keep note of how many gold pieces they have dealt with.

Objective: modify miner.asl specification file in a way that the miner knows, at each point of its execution, how many gold pieces it dropped on the depot.

Test: environment id=3; leader;miner;miner;miner;miner.

Description: the objective can be accomplished by maintaining a belief whose argument is the number of gold pieces dealt with. You should start by creating an initial belief (e.g. **score(0)**). Then, this belief should be updated to reflect the gold pieces successfully dropped on the depot. This can be done as part of the plan to the goal **handle(gold(X,Y))**.

Hints: to update the existing belief **score(X)** with a new score **score(Y)** you can use **-+score(Y)**.

Solution: available [here](#).

d)

Introduction: As you can see, the current implementation of the miner agent has, hardwired, the location of the depot (reflected by the 0,0 in the plans for handling and dropping gold). However, with the exception of the first three scenarios, the depot is not at the location (0,0). In fact, the environment initially sends all agents the location of the depot which is kept as a belief of the form **depot(_X,Y)**, representing that the depot is at location X,Y. We will want the miners to be able to perform in environments where the depot is at an arbitrary location.

Objective: modify the current miner.asl specification file in a way that the miner can operate in environments where location of the depot is not known in advance.

Test: environment id=4; leader;miner;miner;miner;miner

Description: the objective can be accomplished by changing the plans to handle and drop gold so that they check the belief base for the right coordinates for the depot, instead of using (0,0).

Hints: you can include a query to the belief base in the context of the plan, or a test goal, to find out the right coordinates of the depot.

Solution: available [here](#).

Group III

You may have noticed that the multi-agent system has an agent named leader, which has yet to have an active participation. Whereas in a full-fledged implementation of a team of agents it could serve as the coordinator, here we will simply use it to illustrate communication between agents.

e)

Introduction: If you inspect the file leader.asl, you will realize that the leader has a plan to handle messages from the agents informing it that they have dropped gold, keeping every agent's score. According to the plan, every time the leader receives a message from the miners telling it that they dropped some gold at the depot, the leader updates the score of the agent and prints a message. However, right now, the miners are not sending the leader any messages!

Objective: modify the current miner.asl specification file in a way that miners send the leader a message informing it whenever they dropped some gold.

Description: the objective can be accomplished through the modification of the plan to handle gold, adding the action to send the desired message.

Hints: recall the existence of the action **.send(Receiver, Type, Content)** where ToAgent is the name of the destination agent, Type is the kind of message and Content its content (e.g. if Sender sends such a message with **Type=tell**, the effect is the addition of **Content[source(Sender)]** to the belief base of the Receiver, thus generating the corresponding event).

Solution: available [here](#).

f)

Introduction: Despite the fact that the leader keeps the scores, we do not know who is winning.

Objective: modify the current leader.asl specification file in a way that every time there is a new agent leading in number of gold pieces dropped, it prints a message informing who the agent is and how many gold pieces it has dealt with.

Description: the objective can be accomplished through the modification of the leader plan to handle the new belief dropped, considering the case where there is a new winning miner and the case where the winning miner did not change.

Hints: keeping a belief of the form **winning(agent_name,score)** might make it easy to determine when a new message informing of a newly dropped piece of gold causes a change in the winning agent.

Solution: available [here](#).

g)

Introduction: We know (through the console) who is winning, but the agents still do not know.

Objective: modify the current leader.asl specification file in a way that every time there is a new winning agent, it broadcasts a message to every agent.

Description: the objective can be accomplished through the modification of the leader plan to handle the new belief dropped.

Hints: recall the existence of the action **.broadcast(Type, Content)** which acts as the action **.send**, except that the message is sent to every agent.

Solution: available [here](#).

h)

Introduction: Now we want the winning miner to brag about it.

Objective: modify the current miner.asl specification file in a way that every time a miner receives a message telling it that it is the now winning, it prints a message bragging about it.

Description: the objective can be accomplished through the addition of a suitable plan to handle the new belief resulting from the message broadcasted by the leader.

Hints: the internal action `.my_name(X)` instantiates X with the agent's name.

Solution: available [here](#).

Group IV

i)

[Bonus] Introduction: The way the miner.asl specification was implemented is not very efficient because it is too committed to picking up some gold piece, neglecting any gold pieces it finds on its way. It is often the case that a miner walks by a gold piece and doesn't pick it up, simply because it is set to go and pickup another one.

Objective: modify the current miner.asl specification file in a way that miners pick up a gold piece if it is located in its cell, even if it is on its way to picking up another one.

Description: the objective can be accomplished through the addition of a suitable plan to handle the new belief of the form `+cell(X,Y,gold)` when the agent is not carrying gold, but it is also not free.

Hints: this is a more advanced problem. The miner will have to determine its current desire (the internal action

`.desire(handle(gold(X, Y)))` to query the current desires can help) and determine whether the new gold is closer than the one in the current desire. Then, if it is, the miner should drop the current desire (`.drop_desire(handle(gold(X, Y)))`) and adopt a new goal.

Solution: available [here](#).

Acknowledgments: These exercises are based on an implementation of the Gold Miners Scenario developed by Rafael Bordini, Jomi Hubner and Maicon Zatelli.